

HelixCluster

HelixCluster

(repo: helix_cluster · product name “Helix Cluster OS” · Helm chart helix-cluster)

Tagline: A distributed operating system for AI compute — from datacenter GPUs to edge handhelds, under one control plane.

Summary: Helix Cluster OS is a next-generation distributed operating system that orchestrates compute across heterogeneous nodes — from datacenter GPUs down to edge SBCs and handhelds — unifying HPC scheduling, container orchestration, AI/ML inference, federated multi-cluster operation, and secure multi-tenant sessions under a single control plane.

Short description: A Go-based distributed OS / GPU-sharing compute cluster. It unifies HPC scheduling (an Omega-model two-level scheduler), container orchestration, AI inference routing, federation, and secure multi-tenant sessions across heterogeneous nodes, coordinated by SWIM gossip and Raft consensus, with post-quantum end-to-end encryption. (~40 words)

Long description: Helix Cluster OS orchestrates compute workloads across radically heterogeneous hardware — datacenter GPUs, edge single-board computers, even handhelds — under one control plane. It is a Go workspace (monorepo plus git submodules) implementing a seven-layer stack (L0 hardware substrate through L7 federation and observability) with fourteen control-plane microservices. Node membership is tracked with SWIM gossip and discovery; strongly-consistent state uses Raft consensus, organized as per-shard Raft groups with leaseholder-local reads and STONITH fencing. Workload placement runs through an Omega-model two-level scheduler using optimistic concurrency, ClassAd matching, gang scheduling, value-multiplier preemption, and constraint-based placement, extended with carbon-aware and cost/TCO-aware routing, burst-to-cloud autoscaling, and marketplace adapters (Akash, io.net, RunPod, AWS Spot, Chutes). End users interact through a session model (compute allocations), an interactive WebSocket/PTY terminal, an internal AI-inference route, and pool-utilization reads. Security is a first-class layer: SPIFFE identity, device attestation (challenge/

response, proof-of-GPU-work, sealing), an export-control KYC gate, and a post-quantum end-to-end-encrypted transport built on X25519 + ML-KEM-768 hybrid key exchange with AEAD record protection and replay rejection. Correctness is enforced by deterministic simulation testing (FoundationDB-style seeded runs, fault injection, network simulation, byte-for-byte replay, and a Porcupine linearizability checker) and by mandatory paired mutation testing. Architecture and docs are kept honest by mechanical lints that fail the build on drift.

Why we built it: To run AI and HPC workloads across wildly different hardware tiers without stitching together separate schedulers, orchestrators, and inference stacks — and to do it with an engineering guarantee that every shipped feature proves *real end-user behaviour* (never green tests over stubs) and every OS-specific capability uses a real native facility per platform (no Linux-only mocks). The motivating problem, quoted in the repo's governance, is the "tests pass but the feature doesn't actually work" failure mode, which the project is explicitly built to eliminate.

Why it's a game-changer: It collapses HPC scheduling, container orchestration, AI inference, multi-cluster federation, and secure multi-tenant sessions into one control plane spanning datacenter-to-edge hardware — with formal-methods-grade correctness (TLA+ specs, deterministic simulation, linearizability checking) and post-quantum confidential transport usually seen only in specialized systems. Cost- and carbon-aware placement plus cloud-marketplace burst make it economically as well as technically differentiated.

What's innovative: - **Deterministic Simulation Testing (DST)** — seeded, reproducible simulation with fault/clock/network injection, byte-for-byte replay, and a Porcupine linearizability checker. - **Omega-model two-level scheduler** — optimistic concurrency, ClassAd matching, gang scheduling, value-multiplier preemption. - **Post-quantum E2EE / confidential inference wiring** — X25519 + ML-KEM-768 hybrid key exchange, per-request response-keypair binding, AEAD with replay rejection (crypto primitives real/tested; full confidential multi-node round-trip explicitly PLANNED/gated). - **Attestation-driven trust** — SPIFFE identity, proof-of-GPU-work, device sealing, export-control KYC gate, EU AI Act compliance doc-gen. - **Cost- and carbon-aware orchestration** — TCO modeling, carbon-aware placement, burst-to-cloud, N+K failover reserve, cloud-marketplace adapters. - **Multi-Raft consensus** — per-shard Raft groups, leaseholder-local reads, STONITH fencing (IPMI/EC2/Azure/SBD). - **Anti-drift mechanical lints** — archlint fails the build if a documented component's package path doesn't exist; a docs-chain engine keeps Markdown/HTML/PDF/DOCX in sync.

Biggest technical challenges + how solved: - “PASS-bluff” (tests pass on non-functional features). Solved with mandatory paired mutation testing — every work item has a named guard test that must fail under an independent mutation before it can be marked complete. - **Cross-platform parity (no Linux-only mocks).** Solved with one shared interface split by build tags and real per-OS facilities (Linux cgroup//proc/kernel WireGuard; macOS sysctl/vm_stat/IOKit/wireguard-go), tested against an independent OS oracle. - **Distributed correctness under faults.** Solved with deterministic simulation testing and a linearizability checker, plus TLA+ formal specs for consensus/scheduling. - **Documentation and architecture drift.** Solved by archlint (build fails on nonexistent mapped packages) and a docs_chain verify gate with no escape hatch. - **Honest scoping of unfinished work.** Confidential multi-node inference round-trip is explicitly gated behind a ticket and labeled “not yet validated end-to-end” rather than claimed as working.

Tech stack (why + how): - Go (go.mod: 1.25 / toolchain 1.26.4) — primary language for the control plane and ~30-module workspace. - **Zig (0.14+) + C/C++** — low-level system primitives and GPU kernels. - **gRPC + Protocol Buffers** — all inter-subsystem APIs (api/v1/). - **Raft (etcd-raft) + SWIM gossip** — strongly-consistent replicated state + membership/discovery. - **PostgreSQL 16, Redis 7 cluster, etcd v3.5, SQLite** — relational state, cache, coordination, and the local HXC work-item registry. - **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** — messaging/streaming backbones. - **WireGuard mesh + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** — secure, post-quantum transport. - **SPIFFE + JWT (HS256) + scope-based RBAC + OPA** — identity, auth, and policy. - **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, W3C tracing** — observability. - **HashiCorp Vault 1.16** — secrets. - **Docker Compose, Kubernetes (kustomize, hardened securityContext), Helm** — deployment across environments. - **React + TypeScript + Vite (Node 20+)** — web UI. - **TLA+** — formal specification of consensus/scheduling invariants.

Public links: - Product repo, PUBLIC: github.com/HelixDevelopment/helix_cluster - Governance reference: github.com/HelixDevelopment/HelixConstitution - **Note / do not cite as canonical:** the Helm chart lists github.com/HelixCluster/HelixCluster and helixcluster.io, which do NOT match the real remotes — treat these as unverified placeholders. Working remotes are SSH/access-controlled ([github](https://github.com) + gitflic.ru + gitlab.com + gitverse.ru under HelixDevelopment). No public marketing/docs site was found. The bundled LLM stack projects (LLMOrchestrator, LLMProvider, LLMsVerifier) are decoupled submodules, not model servers hosted inside the cluster.

Suggested diagrams/illustrations (OpenDesign): 1. **Helix Cluster architecture** — the seven-layer stack (L0 hardware → L7 federation) with the 14 control-plane microservices, SWIM gossip + Raft consensus, and the Omega scheduler at the core. 2. **Heterogeneous node fabric** — tiers T1-T8 (datacenter GPU → edge SBC → handheld) unified under one control plane, with GPU pools and burst-to-cloud marketplace adapters. 3. **Post-quantum confidential inference path** — X25519 + ML-KEM-768 hybrid handshake → AEAD record protection with replay rejection → attested node (clearly labeling the full round-trip as PLANNED/gated). 4. **Anti-bluff / DST loop** — seeded deterministic simulation with fault injection → replay → Porcupine linearizability check, plus the paired mutation gate required to mark work complete.

Site relevance: Both. Flagship infrastructure product on **vasic.digital** (lead with the distributed-OS + post-quantum + formal-verification story); high-impact portfolio piece on **milosvasic.ru**.

Priority tier: Helix-primary (LLM-infrastructure cluster — the compute substrate that can host inference and compute workloads). Ranks after HelixTrack.

Source provenance: Local repo /Volumes/T7/Projects/helix_cluster — README.md, docs/ARCHITECTURE.md, docs/USER_GUIDE.md, CLAUDE.md, VERSION, deploy/helm/Chart.yaml + values.yaml, deploy/compose/helix_core.yml + helix_infra.yml, deploy/k8s/*.yaml, internal/llm/manager.go, docs/architecture/PHASE_8C_INTEGRATION.md, go.mod/go.work, .gitmodules; cross-checked with harvested _analysis/github-helix-others.md (deep-dive #8) and _analysis/MASTER-inventory.md. Caution flags: version is early (0.1.0-dev); Go version stated inconsistently across docs (README “1.26+”, CLAUDE.md “1.24+”) vs authoritative go.mod (1.25 / toolchain 1.26.4); Helm chart URLs are unverified placeholders; several advanced features (full confidential multi-node inference round-trip, marketplace settlement, attestation-driven scheduling population) are explicitly labeled PLANNED / infra-gated — do NOT present as fully working; coverage figures (main 82.4% / security 87.8%) are self-reported. “Helix-Flow” appears only as an org name in governance boilerplate — not a component.